DistilBERT Model Compression: Pruning, Knowledge Distillation & Quantization

Overview

This notebook demonstrates a complete **model compression pipeline** applied to DistilBERT for news topic classification (AG News dataset). The pipeline has three main stages:

1. **Baseline Training** — Fine-tune a full DistilBERT model as our reference point
2. **Attention Head Pruning + Knowledge Distillation** — Remove unimportant attention heads and retrain with soft targets from the teacher
3. **Dynamic Quantization** — Convert weights to INT8 to further reduce model size and latency

The final comparison shows accuracy, model size (MB), sparsity, and inference latency across all variants.

```
# ============================================================
# STEP 0: Install required libraries
# - transformers: for DistilBERT model and tokenizer
# - datasets: to load AG News from Hugging Face Hub
# - evaluate: for accuracy metric computation
# - accelerate: required backend for Trainer with fp16
# ============================================================
!pip install transformers datasets evaluate accelerate -q
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 0.0/84.1 kB ? eta -:--:--
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 2.7 MB/s eta
0:00:00
```

```
# ============================================================
# STEP 1: Import all necessary libraries
# We use HuggingFace Transformers for the model and training loop,
# PyTorch for tensor ops and device management, and evaluate for
metrics.
# ============================================================
import torch
import numpy as np
import time
import os
from transformers import (
    DistilBertTokenizerFast,
    DistilBertForSequenceClassification,
    TrainingArguments,
    Trainer
)
```

```python
from datasets import load_dataset
import evaluate

# ============================================================
# STEP 2: Select compute device
# If a GPU is available (CUDA), we use it for fast training.
# Otherwise, fall back to CPU. Colab with GPU gives ~10x speedup.
# ============================================================
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")
```

```
Using device: cuda
```

```python
# ============================================================
# STEP 3: Load the AG News dataset
# AG News is a topic classification dataset with 4 categories:
#   0=World, 1=Sports, 2=Business, 3=Sci/Tech
# It contains 120,000 training and 7,600 test samples.
# ============================================================
dataset = load_dataset("ag_news")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/
_auth.py:93: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your
settings tab (https://huggingface.co/settings/tokens), set it as
secret in your Google Colab and restart your session.
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
access public models or datasets.
  warnings.warn(
Warning: You are sending unauthenticated requests to the HF Hub.
Please set a HF_TOKEN to enable higher rate limits and faster
downloads.
WARNING:huggingface_hub.utils._http:Warning: You are sending
unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
enable higher rate limits and faster downloads.
```


```
{"model_id":"54fe9443c50d4eeeae093324f52c32b7","version_major":2,"vers
ion_minor":0}

{"model_id":"11ac5d785b6f47b5b6f7c9479238e901","version_major":2,"vers
ion_minor":0}

{"model_id":"bb3b270637934cc7878d76b915016599","version_major":2,"vers
ion_minor":0}

{"model_id":"9d534c41091f418cbc085d1511c8a17f","version_major":2,"vers
ion_minor":0}
```

{"model_id":"a65f3a37b7f048899432602c4cdc1fcd","version_major":2,"version_minor":0}

```
# Inspect the dataset structure — verifies train/test split sizes
# and confirms the expected features: 'text' and 'label'.
print(dataset)
```

```
DatasetDict({
    train: Dataset({
        features: ['text', 'label'],
        num_rows: 120000
    })
    test: Dataset({
        features: ['text', 'label'],
        num_rows: 7600
    })
})
```

```
# Preview the first training sample to understand the data format:
# each example is a dict with 'text' (news headline) and 'label' (int
0-3).
print(dataset["train"][0])
```

```
{'text': "Wall St. Bears Claw Back Into the Black (Reuters) Reuters -
Short-sellers, Wall Street's dwindling\\band of ultra-cynics, are
seeing green again.", 'label': 2}
```

```
# ============================================================
# STEP 4: Initialize tokenizer
# DistilBertTokenizerFast converts raw text into token IDs.
# We use truncation + padding to a fixed length of 128 tokens
# so all batches have the same shape during training.
# ============================================================
tokenizer = DistilBertTokenizerFast.from_pretrained("distilbert-base-
uncased")
def tokenize(batch):
    return tokenizer(batch["text"], truncation=True,
padding="max_length", max_length=128)
```

{"model_id":"50f17509220f4f40b52297c45f70b6e9","version_major":2,"version_minor":0}

{"model_id":"17df93bc03014d33b73fcb3bc8a3e68a","version_major":2,"version_minor":0}

{"model_id":"dba516ca0fef4b80bf81ef048bdb23c1","version_major":2,"version_minor":0}

```
# ============================================================
# STEP 5: Prepare train/test subsets
# We sample 1,000 examples from each split for faster experimentation.
```

```
# - shuffle(seed=42): reproducible random ordering
# - map(tokenize): applies tokenization to every example in batches
# - set_format('torch'): returns PyTorch tensors instead of Python
lists
# ============================================================
train_data = dataset["train"].shuffle(seed=42).select(range(1000))
test_data  = dataset["test"].shuffle(seed=42).select(range(1000))

train_data = train_data.map(tokenize, batched=True)
test_data  = test_data.map(tokenize, batched=True)

train_data.set_format("torch", columns=["input_ids", "attention_mask",
"label"])
test_data.set_format("torch",  columns=["input_ids", "attention_mask",
"label"])
```

{"model_id":"807c678c3534465b86e8d84d2397ad93","version_major":2,"version_minor":0}

{"model_id":"128f0ba788e54a33b67064c775758ce2","version_major":2,"version_minor":0}

```
# ============================================================
# STEP 6: Define Knowledge Distillation (KD) loss function
# WHY: After pruning, the smaller model needs to recover accuracy.
# Instead of training only on hard labels (one-hot), we use the
# teacher model's soft probability outputs as additional supervision.
#
# The loss is a weighted sum of:
#   - KL Divergence loss (kd_loss): aligns student with teacher's soft
outputs
#   - Cross-Entropy loss (ce_loss): standard classification loss on
ground truth
#
# temperature: 'softens' probability distributions for richer signal
# alpha: controls the balance — higher alpha = more weight on KD loss
# ============================================================
import torch.nn.functional as F

def distillation_loss(student_logits,
                      teacher_logits,
                      labels,
                      temperature=4.0,
                      alpha=0.7):

    kd_loss = F.kl_div(
        F.log_softmax(student_logits / temperature, dim=1),
        F.softmax(teacher_logits / temperature, dim=1),
        reduction="batchmean"
    ) * (temperature ** 2)
```

```python
    ce_loss = F.cross_entropy(student_logits, labels)

    return alpha * kd_loss + (1 - alpha) * ce_loss

# ============================================================
# STEP 7: Compute attention head importance scores
# WHY: We want to identify which attention heads contribute least
# to the model's predictions, so we can safely remove them.
#
# HOW: We pass batches through the model and collect attention
weights.
# The average absolute attention weight per head across batches
# serves as a proxy for how 'active' or 'important' that head is.
# Low-scoring heads are candidates for pruning.
# ============================================================
def compute_head_importance(model, dataloader, n_batches=50):

    head_importance = torch.zeros(
        model.config.num_hidden_layers,
        model.config.num_attention_heads
    ).to(device)

    model.eval()

    for i, batch in enumerate(dataloader):

        if i >= n_batches:
            break

        input_ids = batch["input_ids"].to(device)
        attention_mask = batch["attention_mask"].to(device)

        outputs = model(
            input_ids=input_ids,
            attention_mask=attention_mask,
            output_attentions=True
        )

        attentions = outputs.attentions

        for layer, attn in enumerate(attentions):

            head_importance[layer] += attn.abs().mean(
                dim=(0, 2, 3)
            )

    head_importance /= n_batches

    return head_importance
```

```
# Utility: Measure model file size in megabytes.
# We temporarily save the model's state_dict to disk and measure the
file size.
# This gives us the actual on-disk footprint of the model.
def get_model_size_mb(model):
    torch.save(model.state_dict(), "tmp_model.pt")
    size = os.path.getsize("tmp_model.pt") / (1024 * 1024)
    os.remove("tmp_model.pt")
    return round(size, 2)

# ============================================================
# STEP 8: Define latency and metric utilities
# measure_latency_ms: runs inference n=100 times on GPU and reports
#   average time per forward pass in milliseconds.
#   A warmup phase (10 runs) eliminates JIT/caching artifacts.
# accuracy_metric: loads HuggingFace's accuracy evaluator.
# ============================================================
def measure_latency_ms(model, tokenizer, n=100):
    model.eval()
    text = "Scientists discover new method to improve solar panel
efficiency significantly."
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
                       padding="max_length", max_length=128)
    inputs = {k: v.to(device) for k, v in inputs.items()}
    # Warmup
    with torch.no_grad():
        for _ in range(10):
            model(**inputs)
    # Measure
    start = time.perf_counter()
    with torch.no_grad():
        for _ in range(n):
            model(**inputs)
    elapsed = (time.perf_counter() - start) / n * 1000
    return round(elapsed, 2)

accuracy_metric = evaluate.load("accuracy")
```

{"model_id":"3ca85f7b3d77489a9b73c229c93345d1","version_major":2,"version_minor":0}

```
# compute_metrics: used by HuggingFace Trainer to report accuracy
# at the end of each evaluation epoch. Converts logits -> argmax ->
accuracy.
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    preds = np.argmax(logits, axis=-1)
    return accuracy_metric.compute(predictions=preds,
references=labels)
```

```
# ==============================================================
# STEP 9: Define structured attention head pruning
# WHY: Removing entire attention heads (not just zeroing weights)
# gives real speedup by reducing computation in the attention
mechanism.
#
# HOW:
# - For each layer, rank heads by their importance score (from Step
7).
# - Prune the bottom 60% (prune_ratio=0.6) — the least important
heads.
# - Physically remove the corresponding rows/columns from Q, K, V, and
#   output projection weight matrices.
# - Update attn.n_heads so the model knows the new head count.
# ==============================================================
def prune_heads_by_importance(model, head_importance,
prune_ratio=0.6):
    heads_to_prune = {}
    for layer in range(head_importance.size(0)):
        scores = head_importance[layer]
        n_heads = scores.size(0)
        k = int(n_heads * prune_ratio)
        prune_idx = torch.topk(scores, k,
largest=False).indices.tolist()
        heads_to_prune[layer] = prune_idx

    # Manually prune each transformer layer
    for layer_idx, head_indices in heads_to_prune.items():
        layer = model.distilbert.transformer.layer[layer_idx]
        attn = layer.attention

        n_heads = attn.n_heads
        head_dim = attn.dim // n_heads

        # Build mask of heads to keep
        keep_heads = [i for i in range(n_heads) if i not in
head_indices]
        keep_idx = torch.tensor(
            [h * head_dim + d for h in keep_heads for d in
range(head_dim)],
            dtype=torch.long
        )

        # Prune q, k, v projection weights and biases
        for proj in [attn.q_lin, attn.k_lin, attn.v_lin]:
            proj.weight =
torch.nn.Parameter(proj.weight.data[keep_idx])
            proj.bias   = torch.nn.Parameter(proj.bias.data[keep_idx])

        # Prune output projection (input dim changes)
```

```
        attn.out_lin.weight =
torch.nn.Parameter(attn.out_lin.weight.data[:, keep_idx])

        # Update head count and dim
        attn.n_heads = len(keep_heads)

    total_pruned = sum(len(v) for v in heads_to_prune.values())
    return model, total_pruned

# ==============================================================
# STEP 10: Train the baseline (teacher) model
# WHY: We need a fully trained reference model to:
#   (a) measure baseline accuracy/size/latency before compression
#   (b) serve as the teacher in knowledge distillation later
#
# We fine-tune DistilBERT for 3 epochs with:
#   - batch size 32 (train) / 64 (eval)
#   - fp16 mixed precision for faster GPU training
#   - best checkpoint loaded at the end based on eval loss
# ==============================================================
model_baseline = DistilBertForSequenceClassification.from_pretrained(
    "distilbert-base-uncased", num_labels=4
).to(device)

training_args = TrainingArguments(
    output_dir="./results_baseline",
    num_train_epochs=3,
    per_device_train_batch_size=32,
    per_device_eval_batch_size=64,
    eval_strategy="epoch",
    save_strategy="epoch",
    load_best_model_at_end=True,
    logging_steps=100,
    fp16=True,
)

trainer = Trainer(
    model=model_baseline,
    args=training_args,
    train_dataset=train_data,
    eval_dataset=test_data,
    compute_metrics=compute_metrics,
)

trainer.train()
```

{"model_id":"0a2301f453dd44049c200563fdd85487","version_major":2,"version_minor":0}

{"model_id":"0a8da3ca8af746b7ab8c2972ed891c37","version_major":2,"version_minor":0}

{"model_id":"32fd4edbaf29469498f8bbce15db0a3c","version_major":2,"version_minor":0}

```
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
pre_classifier.weight    | MISSING    |
pre_classifier.bias      | MISSING    |
classifier.bias          | MISSING    |
classifier.weight        | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING  :those params were newly initialized because missing from the checkpoint. Consider training on your downstream task.
```

```
<IPython.core.display.HTML object>
```

{"model_id":"a1e4038b868a4093a6de49184cf31c09","version_major":2,"version_minor":0}

{"model_id":"98ff8bcafc3845078cf84cae5b61d69d","version_major":2,"version_minor":0}

{"model_id":"13a6a9ecd3444ae890e5ae64ab5bb36a","version_major":2,"version_minor":0}

```
There were missing keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.weight', 'distilbert.embeddings.LayerNorm.bias'].
There were unexpected keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.beta', 'distilbert.embeddings.LayerNorm.gamma'].
```

```
TrainOutput(global_step=96, training_loss=0.485763152440389, metrics={'train_runtime': 27.4387, 'train_samples_per_second': 109.335, 'train_steps_per_second': 3.499, 'total_flos': 99354092544000.0, 'train_loss': 0.485763152440389, 'epoch': 3.0})
```

```
# Evaluate and record baseline metrics.
# We capture accuracy, model size (MB), and inference latency (ms)
```

```python
# as the reference numbers to compare against after compression.
# The baseline model is also saved to disk for later use as the KD
teacher.
baseline_acc  = trainer.evaluate()["eval_accuracy"]
baseline_size = get_model_size_mb(model_baseline)
baseline_lat  = measure_latency_ms(model_baseline, tokenizer)

print(f"\n=== BASELINE ===")
print(f"Accuracy : {baseline_acc:.4f}")
print(f"Size     : {baseline_size} MB")
print(f"Latency  : {baseline_lat} ms")

model_baseline.save_pretrained("./baseline_model")
```

```
<IPython.core.display.HTML object>
```

```
=== BASELINE ===
Accuracy : 0.8830
Size     : 255.46 MB
Latency  : 13.07 ms
```

{"model_id":"7b2848b8a4dd49a8a837a6ebe03054e5","version_major":2,"version_minor":0}

```python
# Additional imports for the compression pipeline:
# - DataLoader: to iterate over datasets manually during KD training
# - AdamW: optimizer used for fine-tuning the pruned student model
# - torch.nn.utils.prune: PyTorch's built-in magnitude pruning
utilities
import torch.nn.functional as F
from torch.utils.data import DataLoader
from torch.optim import AdamW
import torch.nn.utils.prune as prune

# Utility: Evaluate model accuracy on CPU.
# Used for the quantized model, which only supports CPU inference.
# Runs on the first n_samples of the test set for quick evaluation.
def evaluate_on_cpu(model, test_data, tokenizer, n_samples=500):
    model.eval()
    correct = 0
    subset = test_data.select(range(n_samples))
    for item in subset:
        inputs = {
            "input_ids":      item["input_ids"].unsqueeze(0),
            "attention_mask": item["attention_mask"].unsqueeze(0),
        }
        with torch.no_grad():
            out = model(**inputs)
        pred = out.logits.argmax(-1).item()
        if pred == item["label"].item():
```

```
            correct += 1
    return correct / n_samples

# Utility: Measure inference latency specifically on CPU.
# Required for the quantized model since dynamic quantization targets
CPU.
# Keeps inputs on CPU (no .to(device)) to match the quantized model's
device.
def measure_latency_cpu(model, tokenizer, n=100):
    """Same as measure_latency_ms but forces CPU — for quantized
model."""
    model.eval()
    text = "Scientists discover new method to improve solar panel
efficiency significantly."
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
                       padding="max_length", max_length=128)
    # inputs stay on CPU — no .to(device)
    with torch.no_grad():
        for _ in range(10):  # warmup
            model(**inputs)
    start = time.perf_counter()
    with torch.no_grad():
        for _ in range(n):
            model(**inputs)
    elapsed = (time.perf_counter() - start) / n * 1000
    return round(elapsed, 2)

# Utility: Move any model to CPU and benchmark latency.
# Used in the final comparison table to ensure all models are
# evaluated on the same device (CPU) for a fair latency comparison.
def measure_latency_cpu_any(model, tokenizer, n=100):
    """Run latency test on CPU for any model."""
    model_cpu = model.cpu()
    model_cpu.eval()
    text = "Scientists discover new method to improve solar panel
efficiency significantly."
    inputs = tokenizer(text, return_tensors="pt", truncation=True,
                       padding="max_length", max_length=128)
    # warmup
    with torch.no_grad():
        for _ in range(10):
            model_cpu(**inputs)
    start = time.perf_counter()
    with torch.no_grad():
        for _ in range(n):
            model_cpu(**inputs)
    elapsed = (time.perf_counter() - start) / n * 1000
    return round(elapsed, 2)
```

```python
# Utility: Compute effective model size accounting for sparsity.
# After magnitude pruning, many weights are zero but still stored.
# This function reports:
#   - file_size: actual disk size
#   - effective_size: size if zero weights were truly removed
(theoretical)
#   - sparsity: percentage of weights that are exactly zero
def get_nonzero_size_mb(model):

    total_params = 0
    nonzero_params = 0

    for p in model.parameters():

        total_params += p.numel()

        nonzero_params += (p != 0).sum().item()

    sparsity = 100 * (1 - nonzero_params / total_params)

    file_size = get_model_size_mb(model)

    effective_size = file_size * (
        nonzero_params / total_params
    )

    return (
        round(file_size,2),
        round(effective_size,2),
        round(sparsity,1)
    )

# Utility: Capture the current sparsity pattern (zero/non-zero mask)
# of all weight tensors. This mask is saved before KD fine-tuning
# so we can re-enforce it after each optimizer step, preventing
# the model from 'filling in' pruned weights during training.
def get_masks(model):

    masks = {}

    for name, param in model.named_parameters():

        if "weight" in name:

            masks[name] = (param != 0).float()

    return masks

# Utility: Re-apply the sparsity mask to all weight tensors.
# Called after every optimizer step during KD training.
# Ensures that weights zeroed by pruning stay zero throughout
```

```
training.
def enforce_masks(model, masks):
    for name, param in model.named_parameters():
        if name in masks:
            param.data *= masks[name].to(param.device)

# ============================================================
# STEP 11: Load baseline model for pruning
# We reload the saved baseline from disk as our starting point.
# 'eager' attention implementation is required for attention head
# pruning to work correctly (vs. the default SDPA implementation).
# ============================================================
from transformers import DistilBertConfig

config = DistilBertConfig.from_pretrained("distilbert-base-uncased")
config._attn_implementation = "eager"
config.num_labels = 4

model_pruned = DistilBertForSequenceClassification.from_pretrained(
    "./baseline_model",
    config=config
).to(device)

print("Model loaded with eager attention.")
```

{"model_id":"a3744fcb624249b3a20be0b79fb29676","version_major":2,"version_minor":0}

```
Model loaded with eager attention.
```

```
# ============================================================
# STEP 12: Apply attention head pruning + magnitude pruning
# Part A — Attention Head Pruning:
#   Compute importance scores on 20 batches, then remove the
#   bottom 60% of heads from every transformer layer.
#
# Part B — FFN / Output Layer Magnitude Pruning:
#   Zero out the smallest weights in the FFN and classification
#   head layers. This creates sparsity without removing structure.
#   We free GPU memory first to avoid OOM during this step.
# ============================================================
import torch.nn.utils.prune as prune
model_baseline.cpu()
torch.cuda.empty_cache()
import gc; gc.collect()
print("Computing head importance scores...")
small_loader = DataLoader(train_data, batch_size=8, shuffle=False)
head_importance = compute_head_importance(model_pruned, small_loader,
n_batches=20)  # was 50
model_pruned, heads_pruned = prune_heads_by_importance(
```

```
    model_pruned, head_importance, prune_ratio=0.6
)

print("\nApplying magnitude pruning to FFN and output layers...")
ffn_layers_pruned = 0
for name, module in model_pruned.named_modules():
    if isinstance(module, torch.nn.Linear):
        if any(x in name for x in ["ffn.lin1", "ffn.lin2",
                                   "attention.out_lin",
                                   "pre_classifier", "classifier"]):
            prune.l1_unstructured(module, name="weight", amount=0.6)
            prune.remove(module, "weight")
            ffn_layers_pruned += 1
            print(f"  Pruned: {name}")

print(f"\nFFN/output layers pruned: {ffn_layers_pruned}")
total = 0; zeros = 0
breakdown = {}
for name, param in model_pruned.named_parameters():
    if "weight" in name:
        z = (param == 0).sum().item()
        n = param.numel()
        total += n
        zeros += z
        if z > 0:
            breakdown[name] = round(z / n * 100, 1)

combined_sparsity = zeros / total * 100
print(f"\nCombined sparsity: {combined_sparsity:.1f}%")
print(f"Zero weights     : {zeros:,} / {total:,}")
print("\nNon-zero sparsity by layer:")
for name, sp in breakdown.items():
    print(f"  {name:<55} {sp}%")

masks = get_masks(model_pruned)
print(f"\nMasks collected: {len(masks)}")
```

```
Computing head importance scores...

Applying magnitude pruning to FFN and output layers...
  Pruned: distilbert.transformer.layer.0.attention.out_lin
  Pruned: distilbert.transformer.layer.0.ffn.lin1
  Pruned: distilbert.transformer.layer.0.ffn.lin2
  Pruned: distilbert.transformer.layer.1.attention.out_lin
  Pruned: distilbert.transformer.layer.1.ffn.lin1
  Pruned: distilbert.transformer.layer.1.ffn.lin2
  Pruned: distilbert.transformer.layer.2.attention.out_lin
  Pruned: distilbert.transformer.layer.2.ffn.lin1
  Pruned: distilbert.transformer.layer.2.ffn.lin2
  Pruned: distilbert.transformer.layer.3.attention.out_lin
```

```
  Pruned: distilbert.transformer.layer.3.ffn.lin1
  Pruned: distilbert.transformer.layer.3.ffn.lin2
  Pruned: distilbert.transformer.layer.4.attention.out_lin
  Pruned: distilbert.transformer.layer.4.ffn.lin1
  Pruned: distilbert.transformer.layer.4.ffn.lin2
  Pruned: distilbert.transformer.layer.5.attention.out_lin
  Pruned: distilbert.transformer.layer.5.ffn.lin1
  Pruned: distilbert.transformer.layer.5.ffn.lin2
  Pruned: pre_classifier
  Pruned: classifier

FFN/output layers pruned: 20

Combined sparsity: 31.1%
Zero weights     : 18,227,409 / 58,646,784

Non-zero sparsity by layer:
  distilbert.transformer.layer.0.attention.out_lin.weight 60.0%
  distilbert.transformer.layer.0.ffn.lin1.weight          60.0%
  distilbert.transformer.layer.0.ffn.lin2.weight          60.0%
  distilbert.transformer.layer.1.attention.out_lin.weight 60.0%
  distilbert.transformer.layer.1.ffn.lin1.weight          60.0%
  distilbert.transformer.layer.1.ffn.lin2.weight          60.0%
  distilbert.transformer.layer.2.attention.out_lin.weight 60.0%
  distilbert.transformer.layer.2.ffn.lin1.weight          60.0%
  distilbert.transformer.layer.2.ffn.lin2.weight          60.0%
  distilbert.transformer.layer.3.attention.out_lin.weight 60.0%
  distilbert.transformer.layer.3.ffn.lin1.weight          60.0%
  distilbert.transformer.layer.3.ffn.lin2.weight          60.0%
  distilbert.transformer.layer.4.attention.out_lin.weight 60.0%
  distilbert.transformer.layer.4.ffn.lin1.weight          60.0%
  distilbert.transformer.layer.4.ffn.lin2.weight          60.0%
  distilbert.transformer.layer.5.attention.out_lin.weight 60.0%
  distilbert.transformer.layer.5.ffn.lin1.weight          60.0%
  distilbert.transformer.layer.5.ffn.lin2.weight          60.0%
  pre_classifier.weight                                   60.0%
  classifier.weight                                       60.0%

Masks collected: 53

# ==============================================================
# STEP 13: Fine-tune pruned model using Knowledge Distillation
# WHY: Pruning causes an accuracy drop. KD helps the pruned (student)
# model recover by mimicking the soft outputs of the teacher.
#
# HOW:
# - Teacher = saved baseline model (frozen, eval mode)
# - Student = pruned model (trainable)
# - Loss = distillation_loss (KL divergence + cross-entropy)
# - After each optimizer step, enforce_masks() re-applies zero mask
```

```
#   so pruned weights are not restored during training.
# ============================================================
# KD training entirely on CPU
kd_device = torch.device("cuda" if torch.cuda.is_available() else
"cpu")
masks = {k: v.to(kd_device) for k, v in masks.items()}
model_baseline = DistilBertForSequenceClassification.from_pretrained(
    "./baseline_model", num_labels=4
).to(kd_device)
model_baseline.eval()

model_pruned = model_pruned.to(kd_device)
model_pruned.train()

optimizer = AdamW(model_pruned.parameters(), lr=2e-5)
train_loader = DataLoader(train_data, batch_size=8, shuffle=True)  #
small batch for CPU
step = 0
masks = get_masks(model_pruned)
print("Starting KD training on CPU...")
for epoch in range(2):
    total_loss = 0
    for batch in train_loader:
        input_ids      = batch["input_ids"].to(kd_device)
        attention_mask = batch["attention_mask"].to(kd_device)
        labels         = batch["label"].to(kd_device)

        with torch.no_grad():
            teacher_out = model_baseline(input_ids=input_ids,
attention_mask=attention_mask)

        student_out = model_pruned(input_ids=input_ids,
attention_mask=attention_mask)
        loss = distillation_loss(student_out.logits,
teacher_out.logits,
                                 labels, temperature=4.0, alpha=0.7)

        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        enforce_masks(model_pruned, masks)

        total_loss += loss.item()
        step += 1
        if step % 50 == 0:
            print(f"  Step {step} | loss: {total_loss/step:.4f}")

    print(f"Epoch {epoch+1} done. Avg loss:
{total_loss/len(train_loader):.4f}")
```

{"model_id":"04efac77b86d4ddd948dd7ced1f6f3ec","version_major":2,"version_minor":0}

```
Starting KD training on CPU...
  Step 50 | loss: 0.9158
  Step 100 | loss: 0.5994
Epoch 1 done. Avg loss: 0.5212
  Step 150 | loss: 0.0228
  Step 200 | loss: 0.0494
  Step 250 | loss: 0.0652
Epoch 2 done. Avg loss: 0.1303
```

```
# Verify that the sparsity pattern was maintained after KD training.
# If sparsity is very low (<20%), it means the mask enforcement failed
# and the model effectively 'undid' the pruning during fine-tuning.
total = 0; zeros = 0
for name, param in model_pruned.named_parameters():
    if "weight" in name:
        total += param.numel()
        zeros += (param == 0).sum().item()
post_sparsity = zeros / total * 100
print(f"\nSparsity after KD training: {post_sparsity:.1f}%")

if post_sparsity < 20:
    print("WARNING: Sparsity too low — mask enforcement may have
failed.")
else:
    print("Sparsity held. Proceeding to evaluation.")
```

```
Sparsity after KD training: 31.1%
Sparsity held. Proceeding to evaluation.
```

```
# Evaluate the pruned + KD model using HuggingFace Trainer.
# This gives us the final accuracy of the compressed model before
quantization.
# The model is then saved to disk for later quantization.
model_pruned.eval()
eval_args    = TrainingArguments(output_dir="./eval_tmp",
                                 per_device_eval_batch_size=64,
                                 do_train=False)
eval_trainer = Trainer(model=model_pruned, args=eval_args,
                       eval_dataset=test_data,
                       compute_metrics=compute_metrics)
pruned_acc   = eval_trainer.evaluate()["eval_accuracy"]
print(f"\nFinal pruned + KD accuracy: {pruned_acc:.4f}")
model_pruned.save_pretrained("./pruned_model")
```

```
<IPython.core.display.HTML object>
```

```
Final pruned + KD accuracy: 0.8720
```

{"model_id":"2b0adcdd41344a80aa6d835a51b21b55","version_major":2,"version_minor":0}

```
# ============================================================
# STEP 14: Apply Dynamic Quantization
# WHY: Quantization converts 32-bit float weights to 8-bit integers
(INT8),
# which reduces model size by ~4x and speeds up CPU inference.
#
# HOW: torch.ao.quantization.quantize_dynamic targets all Linear
layers.
# Dynamic quantization computes activation scales at runtime — no
# calibration dataset needed, making it the simplest quantization
method.
#
# NOTE: Quantized models only run on CPU; we move the model there
first.
# ============================================================
from copy import deepcopy
model_quantized = deepcopy(model_pruned).cpu()
model_quantized = torch.ao.quantization.quantize_dynamic(
    model_quantized, {torch.nn.Linear}, dtype=torch.qint8
)
quant_acc  = evaluate_on_cpu(model_quantized, test_data, tokenizer)
quant_size = get_model_size_mb(model_quantized)
quant_lat  = measure_latency_cpu(model_quantized, tokenizer)

print(f"Quantized — Acc: {quant_acc:.4f} | Size: {quant_size} MB |
Lat: {quant_lat} ms")
```

```
/tmp/ipykernel_590/1902135640.py:14: DeprecationWarning:
torch.ao.quantization is deprecated and will be removed in 2.10.
For migrations of users:
1. Eager mode quantization (torch.ao.quantization.quantize,
torch.ao.quantization.quantize_dynamic), please migrate to use torchao
eager mode quantize_ API instead
2. FX graph mode quantization
(torch.ao.quantization.quantize_fx.prepare_fx,torch.ao.quantization.qu
antize_fx.convert_fx, please migrate to use torchao pt2e quantization
API instead (prepare_pt2e, convert_pt2e)
3. pt2e quantization has been migrated to torchao
(https://github.com/pytorch/ao/tree/main/torchao/quantization/pt2e)
see https://github.com/pytorch/ao/issues/2259 for more details
  model_quantized = torch.ao.quantization.quantize_dynamic(
```

```
Quantized — Acc: 0.8480 | Size: 124.38 MB | Lat: 74.03 ms
```

```
# ============================================================
# STEP 15: Final comparison — Baseline vs Pruned+KD vs Quantized
# We compare all three model variants on the same CPU device for
fairness.
# Metrics reported:
#   - Accuracy   : classification accuracy on the test set
#   - File (MB)  : actual disk size of the saved model
#   - Eff (MB)   : effective size accounting for zero weights
(sparsity)
#   - Sparse (%) : fraction of weights that are exactly zero
#   - Lat (ms)   : average single-sample inference time on CPU
#   - Compress   : compression ratio vs. baseline model size
# ============================================================
bl_lat   = measure_latency_cpu_any(model_baseline, tokenizer)
pr_lat   = measure_latency_cpu_any(model_pruned, tokenizer)

bl_file, bl_eff, bl_sp = get_nonzero_size_mb(model_baseline)
pr_file, pr_eff, pr_sp = get_nonzero_size_mb(model_pruned)

print("\n" + "="*80)
print(f"{'Model':<30} {'Acc':>6} {'File(MB)':>9} {'Eff(MB)':>8}
{'Sparse':>7} {'Lat(ms)':>8} {'Compress':>9}")
print("="*80)
print(f"{'Baseline (DistilBERT)':<30} {baseline_acc:>6.4f}
{bl_file:>9} {bl_eff:>8} {bl_sp:>6}% {bl_lat:>8} {'1.0x':>9}")
print(f"{'+ Pruned + KD':<30} {pruned_acc:>6.4f} {pr_file:>9}
{pr_eff:>8} {pr_sp:>6}% {pr_lat:>8} {bl_eff/pr_eff:>8.1f}x")
print(f"{'+ Pruned + KD + Quant':<30} {quant_acc:>6.4f}
{quant_size:>9} {'N/A':>8} {'N/A':>7} {quant_lat:>8}
{bl_file/quant_size:>8.1f}x")
print("="*80)
```

```
================================================================================
Model                              Acc  File(MB)  Eff(MB)  Sparse
Lat(ms)  Compress
================================================================================
Baseline (DistilBERT)           0.8830    255.46   255.46    0.0%
200.34      1.0x
+ Pruned + KD                   0.8720    223.93   154.38   31.1%
107.16      1.7x
+ Pruned + KD + Quant           0.8480    124.38      N/A     N/A
74.03      2.1x
================================================================================
```